

面向对象程序设计

第八讲 异常

李际军 lijijun@cs.zju.edu.cn



第 8 章 异常

- 本章主要教学内容

- 错误处理逻辑和异常处理的程序思想：异常检查和异常处理的分离
- 异常处理的程序结构
- 异常抛出、捕获和再次抛出
- 异常类、异常参数传递和捕获
- 异常的继承和多态技术

- 本章教学重点

- 异常处理基本思想
- 异常结构和处理流程
- 异常的嵌套处理
- 异常类、异常对象
- 异常捕获的参数（值参和引用参数）
- 异常的继承和多态

- 教学难点

- 异常的参数传递
- 异常的再次抛出和捕获
- 异常对象
- 异常的嵌套处理

第 8 章 异常

1、异常机制解决的主要问题

- 把异常发生与检测处理分离开来，当异常发生时能自动调用异常处理程序进行错误处理，增加了程序错误处理的灵活性。

2、异常机制在软件开发中的作用

- 异常处理机制增加了程序的清晰性和可读性，使程序员能够编写出清晰、健壮、容错能力更强的程序，适用于大型软件开发。

8.1 异常处理概述

1、引入异常机制的原因

- 商业软件常会提供许多错误处理代码，以便对各种可能出现错误的程序代码进行检测和处理，增加程序的健壮性。
- 处理程序错误的代码可能出现在源码的任何地方，比如判定 **new** 是否成功分配内存、数据下标是否越界、运算溢出、除数 **0**、无效参数等。
- 错误处理代码“污染”了程序源码、大量的错误处理代码使原本简单的程序变得晦涩难懂。
- 因此，引入异常机制：将错误检测与处理代码集中管理，以避免错误代码的“污染问题”，使程序更具清晰性、健壮性和灵活性。

8.1 异常处理概述

2、传统的异常处理方法

- 传统程序处理异常的典型方法是不断测试程序继续运行的必要条件，并对测试结果进行处理。
- 形式如下伪码所示：
 - 执行任务 1
 - if 任务 1 未能被正确执行
 - 执行错误处理程序
 - 执行任务 2
 - if 任务 2 未能正确执行
 - 执行错误处理程序
 - 执行任务 3
- 缺点
 - 错误处理代码分布在整个程序的各个部分，使程序受到了错误处理代码的“污染”。

8.1 异常处理概述

3、C++ 异常处理思想

- 其基本思想是将异常发生和异常处理分别放在不同的函数中，产生异常的函数不需要具备处理异常的能力。
- 当一个函数出现异常时，它可以抛出一个异常，然后由该函数的调用者捕获并处理这个异常，如果调用者不能处理，它可以将该异常抛给其上一级的调用者处理。

8.1 异常处理概述

4、C++ 的异常处理的作用

- 改善程序的可读性和可维护性，将异常处理代码与主程序代码分离，适合团队开发大型项目。
- 有力的异常检测和可能的异常恢复，以统一方式处理异常。
- 在异常引起系统错误之前处理异常。
- 处理由库函数引起的异常。
- 在出现无法处理的异常时执行清理工作，并以适当的方式退出程序。
- 在动态调用链中有序地传播异常处理。

8.2 异常处理基础

8.2.1 异常处理的结构

1、异常处理的程序结构

```
try{  
    .....  
    if err1 throw xx1  
    .....  
    if err2 throw xx2  
    .....  
    if errn throw xxn  
}  
catch(type1 arg){.....} // 异常类型 1 错误处理  
catch(type2 arg){.....} // 异常类型 2 错误处理  
catch(typem arg){.....} // 异常类型 m 错误处理  
.....
```

//try 程序块

8.2.1 异常处理的结构

2、try-throw-catch 异常处理的执行流程

- ① 当程序执行过程中遇到 **try** 块时，将**进入 try** 块并按正常的程序逻辑顺序执行其中的语句
- ② 如果 **try** 块的所有语句都被正常执行，**没有发生任何异常**，那么 **try** 块中就**不会有异常被 throw**。在这种情况下，程序将**忽略所有的 catch 块**，顺序执行那些不属于任何 **catch** 块的程序语句，并按正常逻辑结束程序的执行，就像 **catch** 块不存在一样。

8.2.1 异常处理的结构

- 3、如果在执行 **try** 块的过程中，某条语句产生错误并用 **throw** 抛出了异常，则程序控制流程将自此 **throw** 子句转移到 **catch** 块，**try** 块中该 **throw** 语句之后的所有语句都不会再被执行了。
- 4、**C++** 将按 **catch** 块出现的次序，用异常的数据类型与每个 **catch** 参数表中指定的数据类型相比较，如果两者类型相同，就执行该 **catch** 块，同时还将把异常的值传递给 **catch** 块中的形参 **arg**（如果该块有 **arg** 形参）。只要有一个 **catch** 块捕获了异常，其余 **catch** 块都将被忽略。
- 5、如果没有任何 **catch** 能够匹配该异常，**C++** 将调用系统默认的异常处理程序处理该异常，其通常做法是直接终止该程序的运行。

8.2.1 异常处理的结构

【例 8-1】 异常处理的简单例程。

```
#include<iostream>
using namespace std;
void main(){
    cout<<"1--befroe try block..."<<endl;
    try{
        cout<<"2--Inside try block..."<<endl;
        throw 10;
        cout<<"3--After throw ...."<<endl;
    }
    catch(int i) {
        cout<<"4--In catch block1 ... exception..errcode is.."<<i<<endl;
    }
    catch(char * s) {
        cout<<"5--In catch block2 ... exception..errcode is.."<<s<<endl;
    }
    cout<<"6--After Catch...";
}
```

程序运行结果如下：

1--befroe try block...

2--Inside try block...

4--In catch block1 ...errcode is..10

6--After Catch...

8.2.2 异常捕获

1. 异常捕获由 **catch** 完成，**catch** 必须紧跟在与之对应的 **try** 块后面。如果异常被某 **catch** 捕获，程序将执行该 **catch** 块中的代码，之后将继续执行 **catch** 块后面的语句；如果异常不能被任何 **catch** 块捕获，它将被传递给系统的异常处理模块，程序将被系统异常处理模块终止。
 2. **catch** 根据异常的数据类型捕获异常，如果 **catch** 参数表中异常声明的数据类型与 **throw** 抛出的异常的数据类型相同，该 **catch** 块将捕获异常。
- 注意：**catch** 在进行异常数据类型的匹配时，不会进行数据类型的默认转换，只有与异常的数据类型精确匹配的 **catch** 块才会被执行。

8.2.2 异常捕获

【例 8-2】 `catch` 捕获异常时，不会进行数据类型的默认转换。

```
//Eg8-2.cpp
#include<iostream>
using namespace std;
void main(){
    cout<<"1--befroe try block..."<<endl;
    try{
        cout<<"2--Inside try block..."<<endl;
        throw 10;
        cout<<"3--After throw ...."<<endl;
    }
    catch(double i) { // 仅此与例 8.1 不同
        cout<<"4--In catch block1 .. an int type is.."<<i<<endl;
    }
    cout<<"5--After Catch...";
}
```

程序运行结果如下：

1--befroe try block...

2--Inside try block...

abnormal program termination

// 程序因异常而结束

8.3 异常与函数

1. 在函数中处理异常

异常处理可以局部化为一个函数，当每次调用该函数时，异常将被重置。

【例 8-3】 `temperture` 是一个检测温度异常的函数，当温度达到冰点或沸点时产生异常。

```
#include<iostream>
using namespace std;
void temperature(int t){
    try{
        if(t==100) throw " 沸点！ ";
        else if(t==0) throw " 冰点！ ";
        else cout<<"the temperature is OK..."<<endl;
    }
    catch(int x){cout<<"temperatore="<<x<<endl;}
    catch(char *s){cout<<s<<endl;}
}

void main(){
    temperature(0);           //L1
    temperature(10);          //L2
    temperature(100);          //L3
}
```

程序的运行结果如下：
冰点！
the temperature is OK...
沸点！

2. 在函数调用中完成异常处理

- 将产生异常的代码放在一个函数中，将检测处理异常的函数代码放在另一个函数中，能让异常处理更具灵活性和实用性。

【例 8-4】 异常处理从函数中独立出来，由调用函数完成。

```
#include<iostream>
using namespace std;
void temperature(int t) {
    if(t==100) throw " 沸点! ";
    else if(t==0) throw " 冰点! ";
    else cout<<"the temperature is ..."<<t<<endl;
}
void main(){
    try{
        temperature(10);
        temperature(50);
        temperature(100);
    }
    catch(char *s){cout<<s<<endl;}
}
```

程序运行结果如下：

the temperature is ...10
the temperature is ...50
沸点！

8.4 异常处理的几种特殊情况

1. noexcept 异常说明

11C₊₊

– 不抛出异常的方法

- 如果确定某个函数能够正常运行，不会产生任何问题，可以用 **noexcept** 声明它不会产生异常，形式如下：

```
rtype f(.....) noexcept           // 不会抛出异常
{
    .....
}
rtype g(.....)                     // 可能会抛出异常
{
    .....
}
```

- noexcept** 是 C++11 新标准提出的，它与早期版本的空 **throw** 等价，形式如下：

```
rtype f(.....)throw()           // 不会抛出异常
{
    .....
}
```

8.4 异常处理的几种特殊情况

- 关于 **noexcept** 的两点补充

① **noexcept** 说明符实际上可以接受一个 **bool** 类型实参，形式是：

rtype f(.....) noexcept (e) { }

- **e** 是一个逻辑表达式，若其结果为 **true** 表示函数 **f** 不会抛出异常，若结果是 **false** 则表明 **f** 可能会抛出异常。

② **Noexcept** 同时也是一个可以用来判断函数是否会产生异常的运算符，形式为：

noexcept(e) ;

- **e** 是一个表达式，可以包括多个函数调用。例如，对 **f** 和 **g** 函数，
noexcept(f(4)+g(6)) ;

8.4 异常处理的几种特殊情况

2. 捕获所有异常

- 在多数情况下，**catch** 都只用于捕获某种特定类型的异常，但它也具有捕获全部异常的能力。其形式如下：

```
catch(...) {  
    .....  
}
```

// 异常处理代码

【例 8-5】 改写前面的 Errhandler 函数，使之能够捕获所有异常。

```
//Eg8-5.cpp
#include<iostream>
using namespace std;
void Errhandler(int n)throw(){
    try{
        if(n==1) throw n;
        if(n==2) throw "dx";
        if(n==3) throw 1.1;
    }
    catch(...){cout<<"catch an exception..."<<endl;}
}
void main(){
    Errhandler(1);
    Errhandler(2);
    Errhandler(3);
}
```

程序运行结果如下：
catch an exception...
catch an exception...
catch an exception...

8.4 异常处理的几种特殊情况



【例 8-6】 在异常处理块中再次抛出同一异常。

```
#include<iostream>
using namespace std;
void Errhandler(int n){
    try {
        if (n == 1) throw n;
        if (n == 4) throw 'a';
        cout << "all is ok..." << endl;
    }
    catch (int n) {
        n = 100;
        cout << "exception inside is:\t" << n << endl;
        throw;
    }
    catch (char & c) {
        c = 'B';
        cout << "exception inside is:\t" << c << endl;
        throw;
    }
}
```

// 再次抛出 catch

再次抛出的异常将由调用
Errhandler 的函数处理，因此
只能用下面的形式调用本函
数：

```
try{
    Errhandler(...);
    .....
}
catch(...){.....}
```

8.4 异常处理的几种特殊情况

```
void main() {  
    try { Errhandler(1); }  
    catch (int x) { cout << "exception in main...\t" << x << endl; }  
    try { Errhandler(4); }  
    catch (char x) { cout << "exception in main...\t" << x << endl; }  
    cout << "....End..." << endl;  
}
```

程序运行结果如下：
exception inside is: 100
exception in main... 1
exception inside is: B
exception in main... B
....End...

8.4 异常处理的几种特殊情况

4. 异常的嵌套调用

- **try** 块可以嵌套，即一个 **try** 块中可以包括另一个 **try** 块，这种嵌套可能形成一个异常处理的调用链。

【例 8-7】 嵌套异常处理示例。

```
//Eg8-7.cpp
#include<iostream>
using namespace std;
void fc(){
    try{throw "help...";}
    catch(int x){cout<<"in fc..int hanlder"<<endl;}
    try{cout<<"no error handle..."<<endl;}
    catch(char *px){cout<<"in fc..char* hanlder"<<endl;}
}
```

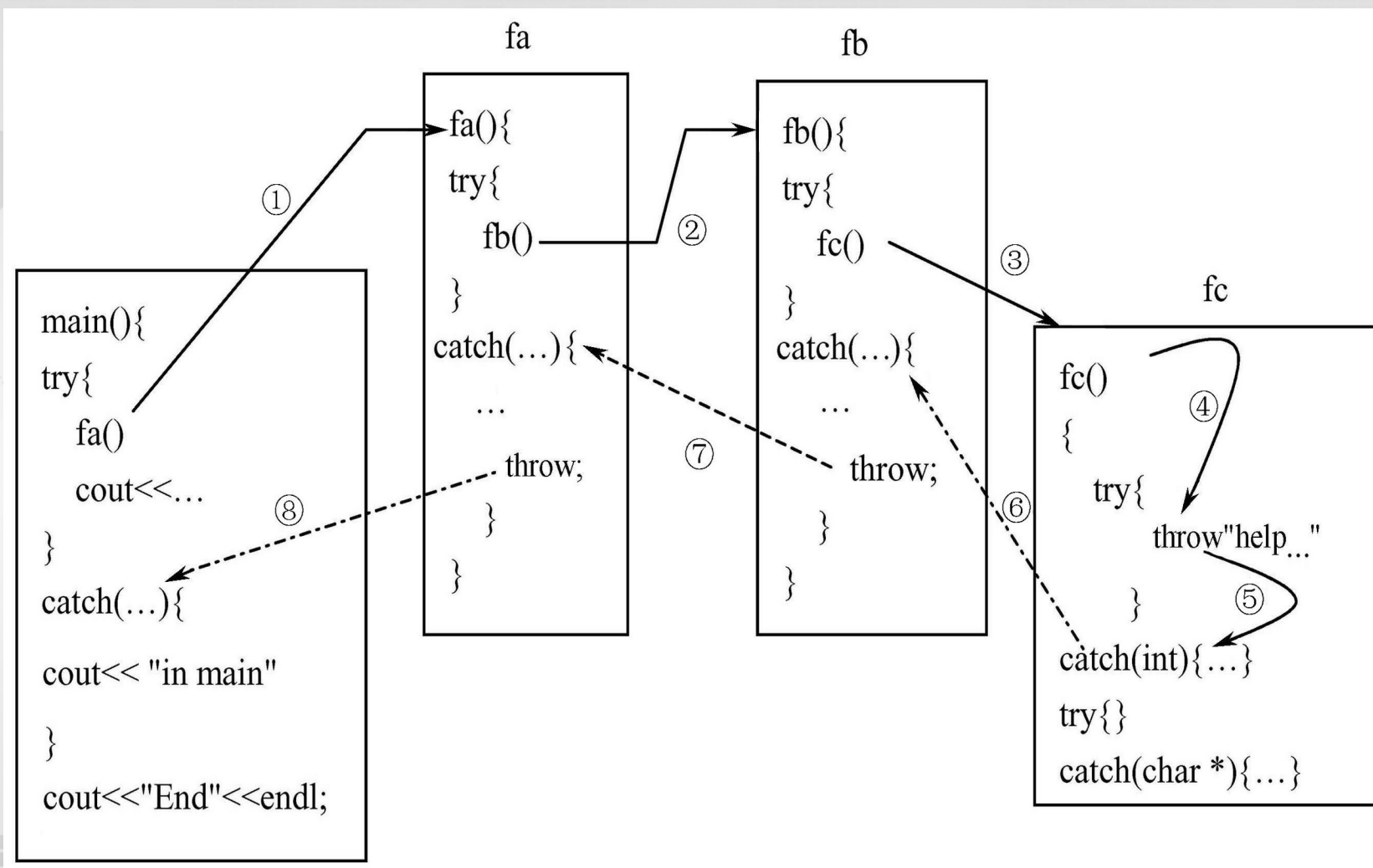


```
void fb(){
    int *q=new int[10];
    try{
        fc();
        cout<<"return form fc()"<<endl;
    }
    catch(...){
        delete []q;
        throw;
    }
}
void fa()
{
    char *p=new char[10];
    try{
        fb();
        cout<<"return from fb()"<<endl;
    }
    catch(...){
        delete []p;
        throw;
    }
}
```

```
void main(){
    try{
        fa();
        cout<<"return from fa"<<endl;
    }
    catch(...){cout<<"in main"<<endl;}
    cout<<"End"<<endl;
}
```

程序运行结果如下：
in main
End

例 8-7 的调用过程



8.5 异常和类

8.5.1 构造函数与异常

- 由于构造函数没有返回类型，在执行构造函数过程中若出现异常，传统处理方法可能是：
 1. 返回一个处于错误状态的对象，外部程序可以检查该对象状态，以便判定该对象是否被成功构造。
 2. 设置一个全局变量保存对象构造的状态，外部程序可以通过该变量值判断对象构造的情况。
 3. 在构造函数中不做对象的初始化工作，而是专门设计一个成员函数负责对象的初始化。
- C++ 中异常处理机制能够很好地处理构造函数中的异常问题，当构造函数出现错误时就抛出异常，外部函数可以在构造函数之外捕获并处理该异常。

8.5 异常和类

【例 8-8】 类 B 有一个类 A 的对象成员数组 obj，类 B 的构造函数进行了自由存储空间的过量申请，最后造成内存资源耗尽，产生异常，则异常将调用对象成员数组 obj 的析构函数，回收 obj 占用的存储空间。

```
//Eg8-8.cpp
#include<iostream>
using namespace std;
class A{
    int a;
public:
    A(int i=0):a(i){}
    ~A(){cout<<"in A destructor..."<<endl;}
};
```

8.5 异常和类

```
class B{
    A obj[3];
    double *pb[10];
public:
    B(int k){
        cout<<"int B constructor..."<<endl;
        for (int i=0;i<10;i++){
            pb[i]=new double[200000000];
            if(pb[i]==0)
                throw i;
            else
                cout<<"Allocated 20000000 doubles in pb["<<i<<"]<<endl;
        }
    };
};

void main(){
    try{B b(2);}
    catch(int e){cout<<"catch an exception when allocated pb["
        <<e<<"]"<<endl; }
}
```

程序运行结果如下:

int B constructor...

Allocated 20000000 doubles in pb[0]

Allocated 20000000 doubles in pb[1]

Allocated 20000000 doubles in pb[2]

Allocated 20000000 doubles in pb[3]

in A destructor...

in A destructor...

in A destructor...

catch an exception when allocated
pb[4]

注意: 本程序异常发生的时间与执行程序
的计算机内存大小密切相关

8.5.2 异常类

1、简单的异常类

- 异常可以是任何类型，包括自定义类。用来传递异常信息的类就是**异常类**。
- 异常类可以非常简单，甚至没有任何成员；
- 异常类也可以同与普通类一样复杂，有自己的成员函数、数据成员、构造函数、析构函数、虚函数等；
- 异常类还可以通过派生方式构成异常类的继承层次结构，实现多态。

【例 8-9】 设计一个堆栈，当入栈元素超出了堆栈容量时，就抛出一个栈满的异常；如果栈已空还要从栈中弹出元素，就抛出一个栈空的异常。

```
//Eg8-9.cpp
#include <iostream>
using namespace std;
const int MAX=3;
class Full{}; //L1 堆栈满时抛出的异常类
class Empty{}; //L2 堆栈空时抛出的异常类
class Stack{
private:
    int s[MAX];
    int top;
public:
    void push(int a);
    int pop();
    Stack(){top=-1;}
};
```

```
void Stack::push(int a){
    if(top>=MAX-1) throw Full();
    s[++top]=a;
}
int Stack::pop(){
    if(top<0) throw Empty();
    return s[top--];
}
void main(){
    Stack s;
    try{
        s.push(10);    s.push(20);    s.push(30);
        //s.push(40); //L5 将产生栈满异常
        cout<<"stack(0)= "<<s.pop()<<endl;
        cout<<"stack(1)= "<<s.pop()<<endl;
        cout<<"stack(2)= "<<s.pop()<<endl;
        cout<<"stack(3)= "<<s.pop()<<endl;        //L6
    }
    catch(Full){    cout<<"Exception: Stack Full"<<endl;    }
    catch(Empty){ cout<<"Exception: Stack Empty"<<endl; }
}
```

程序运行结果如下：

stack(0)= 30

stack(1)= 20

stack(2)= 10

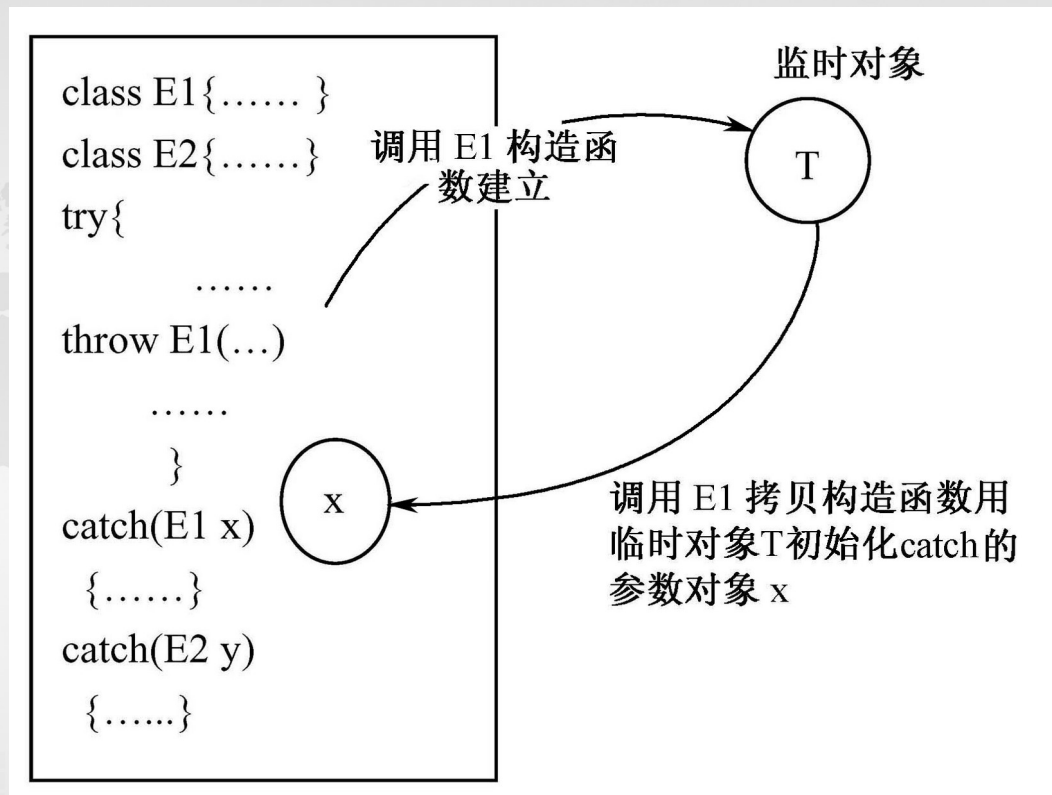
Exception: Stack Empty

8.5.2

异常类

2. 异常对象

- 由异常类建立的对象称为**异常对象**。
- 异常类的处理过程实际上就是异常对象的生成与传递过程。如右图



【例 8-10】修改例 8-9 的 Full 异常类，添加构造函数、成员函数和一个数据成员。利用这些成员，可以获取异常发生时没有入栈的元素信息。

```
//Eg8-10.cpp
#include <iostream>
using namespace std;
const int MAX=3;
class Full{
    int a;
public:
    Full(int i):a(i){}
    int getValue(){return a;}
};
class Empty{};
```

```
class Stack{
private:
    int s[MAX];
    int top;
public:
    Stack(){top=-1;}
    void push(int a){
        if(top>=MAX-1)
            throw Full(a);
        s[++top]=a;
    }
    int pop(){
        if(top<0)
            throw Empty();
        return s[top--];
    }
};
```

```
void main(){
    Stack s;
    try{
        s.push(10);
        s.push(20);
        s.push(30);
        s.push(40);
    }
    catch(Full e){
        cout<<"Exception: Stack
                Full..."<<endl;
        cout<<"The value not push in stack: "
                <<e.getValue()<<endl;
    }
}
```

程序运行结果如下：

Exception: Stack Full...

The value not push in stack: 40

8.5.2 异常类

3. 捕获异常对象的引用

(1) catch 捕获异常的方式

- 按值传递和按引用传递两种方式。

(2) 按值传递异常对象

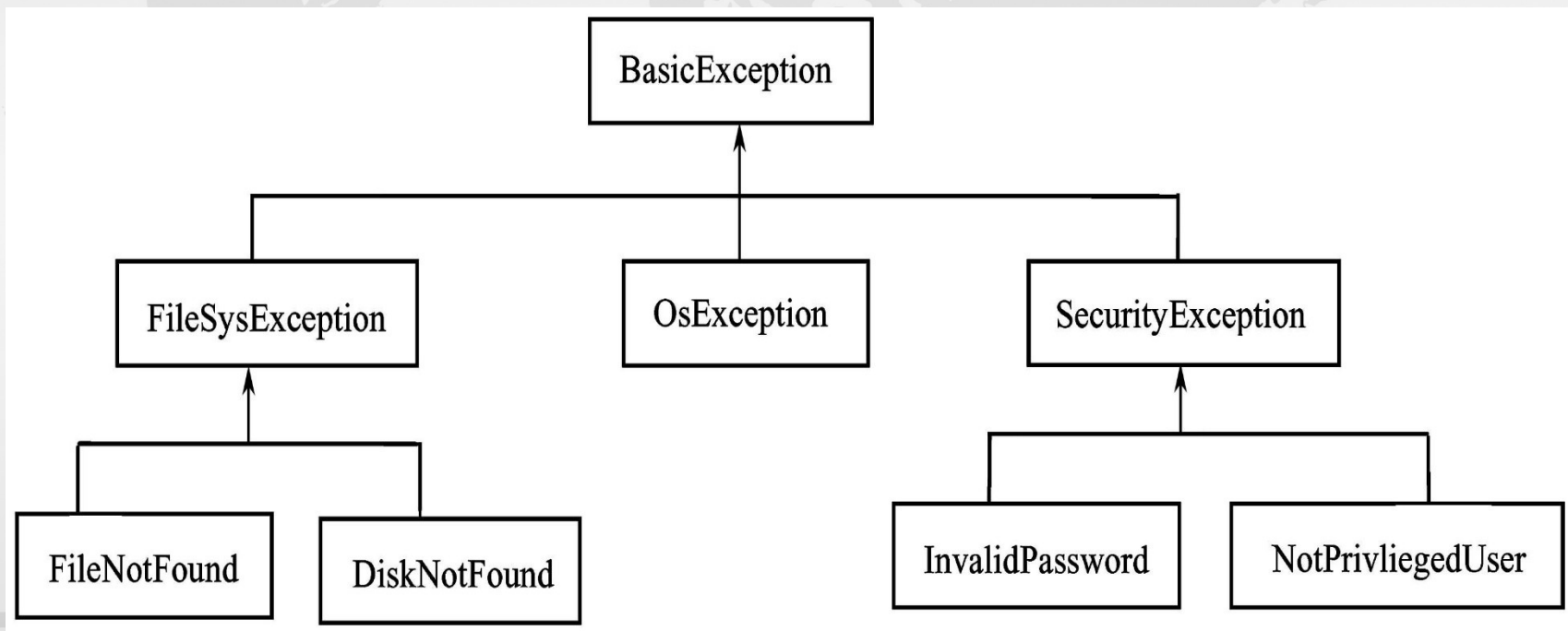
- 如果 catch 块的异常声明是一个值参数，C++ 将以按值传递的方式把 throw 语句抛出的异常对象传递给 catch 参数表中的参数对象。这种方式将调用异常类的拷贝构造函数，用 throw 抛出的临时对象初始化 catch 声明的异常对象。

(3) 按引用传递异常对象

- 如果 catch 块的异常声明是一个引用参数，C++ 将该引用参数绑定到由 throw 抛出的临时异常对象上，即引用参数是临时对象的别名，这种情况不会调用异常类的拷贝构造函数来初始化 catch 参数声明中的异常对象。当异常对象具有较多数据成员时，传引用方式将避免大量的数据拷贝，提高程序效率。

8.5.3 派生异常类的处理

- 在设计软件的异常处理系统时，可以将各种异常汇集起来，根据异常的性质将其分属到不同的类中，形成异常类的继承体系结构。还可以利用类的多态性，将异常类设计为具有多态特性的继承体系，利用多态的强大功能处理异常。
- 一个进行远程登录访问程序的异常类层次结构如图。



【例 8-11】 设计图 8-3 所示异常继承体系中从
BasicException 到 **FileSysException** 部分的异常类。

```
//Eg8-11.cpp
#include<iostream>
using namespace std;
class BasicException{
public:
    char* Where(){return "BasicException...";}
};
class FileSysException:public BasicException{
public:
    char *Where(){return "FileSysException...";}
};
class FileNotFound:public FileSysException{
public:
    char *Where(){return "FileNotFound...";}
};
class DiskNotFound:public FileSysException{
public:
    char *Where(){return "DiskNotFound...";}
};
```

8.5.3 派生异常类的处理

```
void main(){
    try{
        ..... // 程序代码
        throw FileSystemException();
    }
    catch(DiskNotFound p){cout<<p.Where()<<endl;}
    catch(FileNotFound p){cout<<p.Where()<<endl;}
    catch(FileSysException p){cout<<p.Where()<<endl;}
    catch(BasicException p){cout<<p.Where()<<endl;}
    try{
        ..... // 程序代码
        throw DiskNotFound();
    }
    catch(BasicException p){cout<<p.Where()<<endl;}
    catch(FileSysException p){cout<<p.Where()<<endl;}
    catch(DiskNotFound p){cout<<p.Where()<<endl;}
    catch(FileNotFound p){cout<<p.Where()<<endl;}
}
```

程序运行结果如下。

FileSystemException...

BasicException...

其中第 2 行结果是错误的，请分析原因

8.5.3 派生异常类的处理

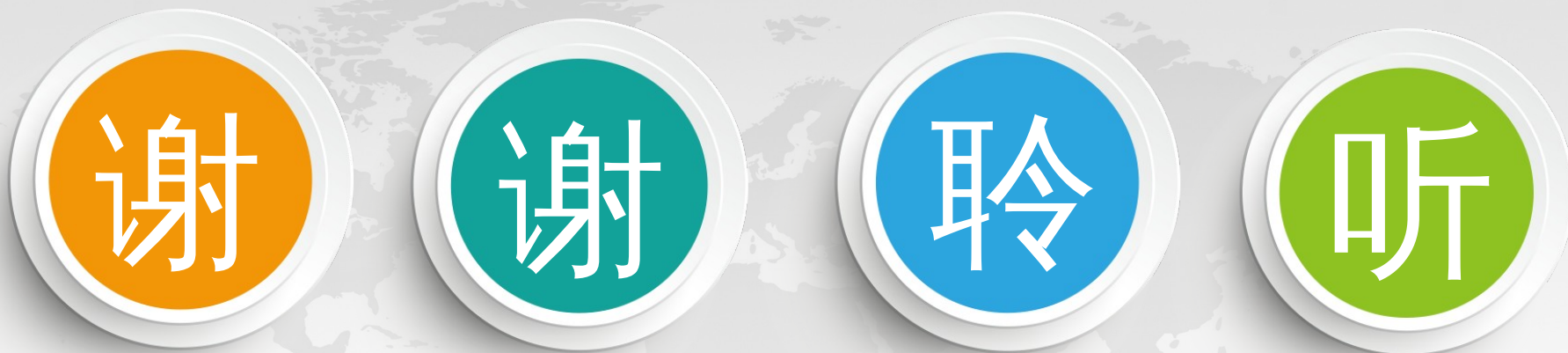


【例 8-12】 设计图 8-3 所示异常继承体系中从
BasicException 到 FileSysException 部分的多态异常类。

```
#include <iostream>
using namespace std;
class BasicException{
public:
    virtual char* Where(){return "BasicException...";}
};
.....
void main(){
    try{
        //..... 程序代码
        throw FileSysException();
    }
    catch(BasicException &p){cout<<p.Where()<<endl;}
    try{
        //..... 程序代码
        throw DiskNotFound();
    }
    catch(BasicException &p){cout<<p.Where()<<endl;}
}
```

运行结果。
FileSysException...
DiskNotFound...

不仅结果正确，而且代码简洁，
此多态的效能也！



THANKS!